ABSTRACT

Accurate depth estimation is a fundamental requirement for autonomous driving systems that must navigate complex and dynamic environments safely. Conventional autonomous vehicles rely on LiDAR sensors to acquire precise depth measurements; however, these sensors are prohibitively expensive, ranging from five thousand to seventy-five thousand dollars per unit, significantly limiting the scalability and mass-market deployment of intelligent transportation systems. This constraint has motivated extensive research into monocular depth estimation, which aims to infer scene depth from a single RGB camera frame using advanced deep learning techniques.

This project presents a real-time monocular depth estimation system for autonomous driving based on the Monodepth2 self-supervised architecture integrated with the CARLA autonomous driving simulator. The proposed system employs a ResNet encoder backbone for hierarchical feature extraction, combined with a multi-scale depth decoder that generates dense depth maps without requiring ground-truth LiDAR annotations during training. Self-supervised training is achieved through photometric consistency loss, which leverages temporal frame sequences to optimize depth and pose predictions jointly.

The system pipeline encompasses CARLA simulation environment configuration, real-time RGB frame acquisition at ten frames per second, preprocessing including resizing and normalization, depth inference using pretrained Monodepth2 weights, and visual overlay of colorized depth maps onto live simulation frames. Evaluation is performed using standard monocular depth metrics including Absolute Relative Error, Root Mean Square Error, Squared Relative Error, delta accuracy thresholds, and Log RMSE, targeting an Absolute Relative Error below 0.115 and RMSE below 4.8 meters on KITTI benchmark evaluations.

The framework is designed for deployment on commodity hardware equipped with standard GPU acceleration, democratizing autonomous vehicle research by eliminating dependence on expensive sensor arrays.

TABLE OF CONTENTS

SECTION	TITLE	PAGE NO.
Chapter 1	Introduction	1 – 5
Chapter 2	Literature Survey	6 - 12
Chapter 3	Problem Statement, Objectives and Scope	13 - 15
Chapter 4	Proposed Methodology and System Design	16 - 19
Chapter 5	Dataset / Data Collection and Feasibility	20 - 22
Chapter 6	Evaluation Plan	23 - 24
Chapter 7	Work Plan and Phase-II Timeline	25
Chapter 8	Conference Paper Status	26
Chapter 9	Conclusion and Future Work	27 - 28
References	IEEE Format	29 - 30

CHAPTER 1: INTRODUCTION

1.1 Background

This project lies in the domain of Autonomous Perception and Computer Vision, specifically focusing on the problem of monocular depth estimation for autonomous driving applications. Depth perception is the ability of a system to determine the distance of objects in its environment, which is a foundational capability for safe autonomous navigation, obstacle detection, and collision avoidance. In conventional autonomous driving platforms, depth is acquired using LiDAR (Light Detection and Ranging) sensors, which emit laser pulses and measure the time-of-flight of reflections to generate highly accurate three-dimensional point clouds.

While LiDAR-based depth acquisition is accurate, the associated hardware costs—ranging from five thousand dollars for low-resolution units to over seventy-five thousand dollars for high-resolution automotive-grade systems—present a substantial barrier to the widespread deployment of autonomous vehicles. This has motivated significant research interest in camera-based alternatives, particularly monocular depth estimation, wherein depth information is inferred from a single RGB camera frame using deep neural networks.

The advent of deep learning has transformed the landscape of monocular depth estimation. Early CNN-based approaches introduced supervised training with LiDAR-annotated ground truth, demonstrating that neural networks could learn to predict depth from visual texture, perspective, and contextual cues. Subsequent developments introduced self-supervised training paradigms, which eliminated the need for expensive LiDAR annotations by leveraging photometric consistency across sequential video frames as a training signal. Architectures such as Monodepth2, developed by Godard et al. in 2019, established the benchmark for self-supervised monocular depth estimation and remain widely used as a foundation for subsequent research.

This project builds upon the Monodepth2 framework and integrates it with the CARLA open-source autonomous driving simulator to create a complete real-time depth estimation pipeline. The CARLA simulator provides a rich, configurable urban driving environment with ground-truth depth sensing capability, enabling controlled evaluation of the depth estimation system across diverse driving scenarios.

1.2 Motivation

The motivation for this project arises from a convergence of technical, economic, and social factors that highlight the critical need for scalable depth estimation solutions in autonomous driving. From a purely economic standpoint, the prohibitive cost of LiDAR sensors severely restricts autonomous vehicle development to a small number of well-funded research institutions and large automotive corporations. A camera-only depth estimation system capable of achieving comparable performance at a fraction of the cost would democratize autonomous vehicle research, enabling smaller institutions and developing nations to contribute to this transformative technology.

From a technical standpoint, monocular depth estimation remains an open research problem characterized by several persistent challenges. Current state-of-the-art methods achieve strong performance on established benchmarks such as KITTI, yet struggle to generalize to new environments, lighting conditions, and dynamic driving scenarios. Self-supervised methods like Monodepth2, while annotation-efficient, suffer from inherent scale ambiguity and reduced accuracy on dynamic objects, which violate the static scene assumptions underlying photometric consistency loss computation.

The social impact of improved autonomous depth estimation is equally compelling. According to the World Health Organization, approximately 1.35 million people die annually in road traffic accidents, the majority of which involve human error. Reliable autonomous perception systems capable of accurate real-time depth estimation could significantly reduce these fatalities by enabling safer, more reliable autonomous driving. Furthermore, making autonomous vehicle technology accessible through cost-effective camera-based solutions could accelerate deployment in public transportation and logistics sectors, yielding broader societal benefits.

From an academic perspective, this project explores the intersection of self-supervised deep learning, computer vision, and autonomous systems—an area that is rapidly evolving with significant open research questions. The integration of Monodepth2 with CARLA provides an opportunity to investigate domain adaptation between synthetic simulation environments and real-world driving data, a critical challenge for deploying learned perception systems in practical autonomous vehicles.

1.3 Need for the Project

Existing approaches to depth estimation in autonomous driving can be broadly categorized into sensor-based methods and learning-based methods. Sensor-based approaches relying on LiDAR achieve high accuracy but suffer from the cost, weight, and maintenance limitations described above. Stereo camera systems offer a relatively cost-effective alternative and provide metric-scale depth estimation, but require precise calibration, suffer from performance degradation in low-texture regions, and increase system complexity through the use of dual synchronized cameras.

Among learning-based approaches, supervised monocular depth estimation methods trained on LiDAR-annotated datasets achieve competitive accuracy but require expensive data collection infrastructure and do not generalize well to unseen environments. Self-supervised methods, while eliminating annotation requirements, present their own challenges including scale ambiguity, poor performance on dynamic objects, and sensitivity to illumination changes. Furthermore, many state-of-the-art models are computationally intensive and cannot achieve real-time inference on embedded hardware without specialized optimization.

The need for this project therefore arises from the gap between the theoretical promise of monocular depth estimation and its practical deployability in autonomous driving scenarios. By implementing a complete end-to-end pipeline from simulation data collection through real-time depth inference and visualization, this project provides a validated, reproducible system that

demonstrates the feasibility of camera-only depth estimation for autonomous driving applications on commodity hardware.

1.4 Project Overview

The proposed system is a real-time monocular depth estimation framework that integrates the Monodepth2 self-supervised deep learning architecture with the CARLA autonomous driving simulator. The system captures RGB video frames from a simulated vehicle camera, processes them through a ResNet encoder and multi-scale depth decoder, and generates dense colorized depth maps overlaid on the original frames in real time. The pipeline is designed to operate at ten frames per second on standard GPU hardware, making it suitable for real-time autonomous driving applications.

The major modules of the system include a Simulation Module responsible for CARLA environment configuration and sensor data acquisition, a Data Pipeline Module for frame preprocessing and augmentation, a Model Module implementing the Monodepth2 ResNet encoder and depth decoder architecture, and a Visualization Module for real-time depth map rendering and performance metric display. Together, these modules constitute a complete and validated autonomous perception system.

The primary expected users of this system include autonomous vehicle researchers evaluating depth estimation algorithms, robotics engineers developing perception stacks for autonomous platforms, and academic researchers conducting studies in self-supervised learning and domain adaptation. The system is designed to be modular and extensible, facilitating future integration with object detection, semantic segmentation, and motion planning modules.

1.5 Organization of the Report

Chapter 1: Introduction introduces the project domain, background, and real-world context. It explains the importance of monocular depth estimation in autonomous driving and outlines the motivation, need, and overview of the proposed work.

Chapter 2: Literature Survey reviews existing research and techniques related to monocular depth estimation, self-supervised learning, and autonomous perception. It highlights key findings, limitations, and research gaps from the surveyed literature.

Chapter 3: Problem Statement, Objectives and Scope formally define the core problem being addressed, states the specific objectives of the project, and specifies the scope including what is covered and what lies beyond the project boundary.

Chapter 4: Proposed Methodology and System Design describes the complete system architecture including the Monodepth2 pipeline, CARLA integration, data preprocessing, ResNet encoder, depth decoder, and self-supervised loss formulation.

Chapter 5: Dataset / Data Collection and Feasibility explains the datasets used, data collection procedures, preprocessing pipeline, hardware and software requirements, and a comprehensive feasibility analysis.

Chapter 6: Evaluation Plan outlines the quantitative metrics including Absolute Relative Error, RMSE, and delta accuracy thresholds, along with the evaluation methodology and target performance values.

Chapter 7: Work Plan and Phase-II Timeline provides a detailed Gantt chart and milestone-based plan for Phase-II implementation activities.

Chapter 8: Conference Paper Status describes the progress and status of the research paper associated with this project.

Chapter 9: Conclusion and Future Work summarizes the Phase-I findings and outlines research directions for future improvements.

References lists all research papers and resources in IEEE citation format. The Appendix includes supplementary materials such as presentation slides and additional diagrams.

CHAPTER 2: LITERATURE SURVEY

2.1 Literature Collection Strategy

The literature survey for this project was conducted using reputed academic databases including IEEE Xplore, ACM Digital Library, SpringerLink, Elsevier ScienceDirect, MDPI, and arXiv, ensuring comprehensive coverage of peer-reviewed and preprint research in monocular depth estimation and autonomous driving. These sources provide reliable and high-quality publications spanning computer vision, deep learning, and autonomous systems.

Keywords used for literature search included: "monocular depth estimation", "self-supervised depth estimation", "autonomous driving perception", "Monodepth2", "ResNet depth decoder", "photometric consistency loss", "KITTI depth benchmark", "CARLA simulator depth", "transformer depth estimation", and "real-time depth inference". Boolean combinations of these keywords were used to refine search results to the most relevant publications.

Inclusion criteria for paper selection required publications between 2014 and 2025 from reputed journals and conferences such as IEEE CVPR, ICCV, NeurIPS, ECCV, and IEEE Transactions on Intelligent Transportation Systems, with strong relevance to monocular depth estimation, self-supervised learning, or autonomous perception. A total of twenty-five papers were reviewed and analyzed for this survey.

2.2 Literature Matrix

#	Paper Title+Year	Authors	Methods	Dataset	Result	Limitation	Research Gap
1	Depth Map Prediction from a Single Image using Multi-Scale Deep Network (2014)	Eigen et al.	Two-stack CNN: coarse global + fine local refinement; scale-invariant loss	NYU-v2, KITTI	AbsRel 0.215 NYU, RMSE 7.216 KITTI	Blurry boundaries; no temporal context; limited resolution	No real-time inference; poor edge sharpness on thin structures
2	Unsupervised CNN for Single View Depth Estimation (2016)	Garg et al.	Unsupervised CNN with image reconstruction loss from stereo pairs	KITTI Eigen	AbsRel 0.169; first viable unsupervised depth training paradigm	Requires stereo pairs at training; not purely monocular	Cannot generalize to monocular-only pipelines
3	Deeper Depth Prediction with Fully Convolutional Residual Networks (2016)	Laina et al.	FCRN: ResNet50 encoder with up-projection decoder blocks	NYU-v2, Make3D	AbsRel 0.127 NYU; RMSE 0.573; SOTA at publication time	Fixed resolution; slow inference for real-time use	Lacks temporal consistency; no dynamic scene handling
4	Digging Into Self-Supervised Monocular Depth Estimation — Monodepth2 (2019)	Godard et al.	Min-reprojection loss, auto-masking, multi-scale ResNet encoder-decoder	KITTI Eigen	AbsRel 0.115; backbone for many subsequent works	Scale ambiguity; struggles with dynamic objects and textureless regions	No moving vehicle handling; scale non-recoverable without additional sensor
5	BTS: Local Planar Guidance for	Lee et al.	Local planar guidance decoder layers;	KITTI, NYU-v2	AbsRel 0.061 KITTI; RMSE 2.605; SOTA	Supervised; requires dense	Not suited for self-supervised or

	Monocular Depth Estimation (2019)		DenseNet/ResNet encoder with planar constraints		supervised 2019	LiDAR annotations; high parameter count	label-scarce driving scenarios
6	PackNet-SfM: 3D Packing for Self-Supervised Depth Estimation (2020)	Guizilini et al.	3D packing/unpacking convolutions; velocity supervision for metric scale	KITTI, nuScenes	AbsRel 0.111; first self-supervised method with metric scale recovery	Slow inference due to 3D convolutions; not lightweight for edge deployment	Computational overhead limits real-time autonomous driving use
7	AdaBins: Depth Estimation Using Adaptive Bins (2021)	Bhat et al.	Encoder-decoder + mini-ViT predicting adaptive depth bin widths per image	NYU-v2, KITTI	AbsRel 0.058 NYU and KITTI; SOTA at CVPR 2021	Fixed encoder; mini-ViT adds inference latency; not self-supervised	Requires dense ground truth depth annotations for training
8	Vision Transformers for Dense Prediction (DPT) (2021)	Ranftl et al.	ViT encoder reassembling multi-stage tokens into multi-resolution features	NYU-v2, KITTI, MIX-6	28% relative improvement over MiDaS; SOTA zero-shot transfer	Large model size; high GPU memory; slow for real-time driving	Not optimized for speed; impractical for embedded automotive hardware
9	Self-Supervised MDE: Solving the Dynamic Object Problem (2021)	Klingner et al.	Semantic segmentation-guided masking of dynamic objects in photometric loss	KITTI, Cityscapes	AbsRel 0.099; dynamic mask reduces artifacts by 15%	Requires pretrained segmentation model; adds pipeline complexity	Weak masks from poor detector degrade performance in crowded scenes
10	Lite-Mono: Lightweight	Zhang et al.	Hybrid CNN+Transformer	KITTI, Make3D	AbsRel 0.107; 3M params;	Reduced accuracy on	Performance gap in night,

	CNN-Transformer for Self-Supervised Depth (2023)		encoder; multi-scale local-global feature fusion		20+ FPS on embedded GPU	challenging scenes vs. larger models	rain, and adverse weather conditions
11	Metric3D: Zero-Shot Metric 3D Prediction from Single Image (2023)	Yin et al.	Large-scale multi-dataset training with canonical camera space transformation; ViT backbone	KITTI, NYU, 8+ datasets	SOTA zero-shot metric depth on KITTI (delta1=0.964) and NYU	Complex training pipeline; requires camera intrinsics; struggles with wide-angle cameras	Camera configuration dependency limits universal deployment
12	Depth Anything: Power of Large-Scale Unlabeled Data (2024)	Yang et al.	Semi-supervised on 62M unlabeled images; DINOv2 encoder; semantic supervision	KITTI, NYU, 6+ zero-shot	SOTA on 6 zero-shot benchmarks; strong generalization	Relative depth only in base model; requires fine-tuning for metric scale	Metric depth needs additional calibration for navigation use
13	Depth Anything V2 (2024)	Yang et al.	Synthetic data training + large teacher + pseudo-labeled real image distillation	KITTI, NYU, DA-2K	Significantly finer depth predictions; 10x faster than diffusion methods	Synthetic-to-real domain gap; limited scene diversity in synthetic data	Nighttime and extreme weather generalization remains unaddressed
14	LRDepth: High-Frequency and Diffusion Refinement for Depth (2025)	Zhang et al.	HFEM high-frequency enhancement + progressive residual denoising diffusion	KITTI, DIODE	Improved long-range depth accuracy and edge detail; competitive KITTI SOTA	Diffusion module increases inference latency; not real-time capable	Diffusion steps incompatible with real-time autonomous driving requirements
15	Metric3D v2: Versatile	Hu et al.	ViT-giant backbone; joint	KITTI, NYU,	Rank 1 KITTI and NYU	Extremely large model;	Real-time inference

	Monocular Geometric Foundation Model (2024)		depth+normal training on 16M+ images	ETH3D, SUN RGB-D	benchmarks; SOTA zero-shot metric depth	high compute requirement; not real-time deployable	requires dedicated accelerators unavailable in commodity hardware

2.3 Summary of Literature Survey

The literature survey reveals that monocular depth estimation for autonomous driving has undergone rapid evolution since the introduction of deep convolutional neural networks for this task in 2014. Early supervised approaches demonstrated the viability of depth prediction from single RGB images but depended on expensive LiDAR annotations. Self-supervised methods, beginning with stereo-based reconstruction and progressing to fully monocular temporal consistency approaches epitomized by Monodepth2, significantly reduced annotation requirements while maintaining competitive performance.

Recent developments have been characterized by the integration of Vision Transformer architectures, which provide enhanced global contextual understanding but at substantially higher computational cost. Hybrid CNN-transformer designs such as Lite-Mono represent an effort to balance accuracy and efficiency for real-time deployment scenarios. Foundation model approaches including Depth Anything and Metric3D have demonstrated remarkable zero-shot generalization through large-scale training but remain impractical for embedded automotive hardware due to model size and latency constraints.

Common evaluation benchmarks across the surveyed literature include the KITTI Eigen split and NYU Depth V2, with Absolute Relative Error and delta accuracy thresholds serving as primary performance metrics. Key findings indicate that self-supervised methods consistently achieve Absolute Relative Error in the range of 0.099 to 0.115 on KITTI, while supervised methods achieve

values below 0.065. The gap between self-supervised and supervised performance narrows with architectural improvements and auxiliary training signals.

2.4 Identified Research and Application Gaps

Based on the comprehensive literature survey, the following key research and application gaps have been identified that motivate the proposed work:

- Existing self-supervised models including Monodepth2 suffer from scale ambiguity, making metric depth estimation impossible without additional calibration inputs.
- Current high-accuracy depth estimation models such as Depth Anything V2 and Metric3D v2 are computationally expensive and cannot achieve real-time inference on commodity automotive hardware.
- The generalization of self-supervised depth models from synthetic simulation environments such as CARLA to real-world driving datasets remains an open research challenge.
- Dynamic object handling remains problematic in photometric-consistency-based self-supervised methods, which assume a static world during training.
- Limited prior work has systematically evaluated Monodepth2 in an integrated real-time pipeline with the CARLA simulator, particularly with respect to domain adaptation between synthetic and real-world data.
- Robustness of depth estimation under adverse driving conditions including low light, rain, and fog has not been adequately addressed in practical real-time systems.

CHAPTER 3: PROBLEM STATEMENT, OBJECTIVES AND SCOPE

3.1 Problem Statement

Autonomous vehicles require accurate, real-time depth perception to navigate safely in complex urban environments. Current industrial autonomous platforms predominantly rely on LiDAR sensors, which provide high-precision three-dimensional point cloud data but cost between five thousand and seventy-five thousand dollars per unit. This cost barrier renders mass-market autonomous vehicle deployment economically infeasible and concentrates autonomous driving research within a small number of resource-rich organizations.

Monocular depth estimation offers a promising low-cost alternative, using a single RGB camera and deep learning to infer scene depth without specialized ranging hardware. However, existing monocular depth estimation models suffer from critical limitations including poor generalization to unseen driving environments, vulnerability to dynamic objects that violate photometric consistency assumptions, scale ambiguity in self-supervised training, and insufficient real-time inference performance for practical deployment. Furthermore, most existing works have not been rigorously evaluated in controlled simulation environments that mirror real-world autonomous driving conditions.

This project addresses these limitations by implementing a complete, validated, real-time monocular depth estimation pipeline based on the Monodepth2 self-supervised architecture, integrated with the CARLA autonomous driving simulator for controlled data collection and evaluation. The system generates dense depth maps from single RGB camera frames at real-time inference speeds, enabling practical deployment on commodity GPU hardware without requiring expensive sensor infrastructure.

3.2 Objectives

The specific objectives of this project are as follows:

1. Build and fine-tune the Monodepth2 self-supervised architecture for monocular depth prediction from RGB images, leveraging pretrained KITTI weights as initialization for improved convergence.
2. Integrate the depth model with the CARLA autonomous driving simulator to process real-time camera feed data from a simulated vehicle navigating urban environments.
3. Develop a preprocessing and inference pipeline that generates and displays colorized depth maps in real time as the simulated vehicle navigates, achieving at least ten frames per second.
4. Evaluate model performance using standard monocular depth metrics including Absolute Relative Error, RMSE, Squared Relative Error, delta accuracy thresholds, and Log RMSE, against KITTI ground truth and CARLA synthetic depth data.
5. Ensure the complete system operates efficiently on commodity hardware equipped with a mid-range GPU, without requiring specialized sensor infrastructure or proprietary hardware accelerators.

3.3 Scope of the Project

3.3.1 In Scope

- Monocular depth estimation using the Monodepth2 self-supervised deep learning architecture with ResNet encoder and multi-scale depth decoder.
- Integration with CARLA autonomous driving simulator version 0.9.13 using the Python CARLA API for vehicle spawning, camera sensor configuration, and real-time frame acquisition.
- Real-time depth map generation from single RGB camera frames at a target rate of ten frames per second using GPU-accelerated PyTorch inference.
- Evaluation on CARLA synthetic data and KITTI benchmark dataset using standard monocular depth metrics.
- Performance analysis under standard lighting conditions in urban driving scenarios within the CARLA simulation environment.

- Lightweight deployment on commodity hardware including Intel or AMD CPUs and NVIDIA mid-range GPUs with at least four gigabytes of VRAM.
- Data preprocessing pipeline including frame resizing, normalization, augmentation, and temporal triplet grouping for self-supervised pose estimation.

3.3.2 Out of Scope

- Real-world physical vehicle deployment, hardware integration, or on-road testing of the system.
- Stereo or multi-camera depth estimation systems requiring synchronized multi-sensor calibration.
- Night-time or adverse weather performance optimization, including rain, fog, and snow conditions.
- End-to-end autonomous driving control policy including path planning, steering, and throttle control.
- Multi-modal sensor fusion incorporating LiDAR, radar, or GPS alongside camera inputs.
- Cloud-based or distributed deployment architectures for fleet-scale autonomous vehicle applications.

CHAPTER 4: PROPOSED METHODOLOGY AND SYSTEM DESIGN

4.1 System Architecture Overview

The proposed system follows a four-stage deep learning pipeline designed for real-time monocular depth estimation in autonomous driving simulation environments. The pipeline proceeds from the CARLA Simulator through a Data Preprocessing Module, the Monodepth2 Model (comprising ResNet Encoder and Depth Decoder), and concludes with a Visualization and Evaluation Module. Each stage is designed for modularity, allowing individual components to be replaced or upgraded without disrupting the overall pipeline.

The architectural design prioritizes real-time inference performance while maintaining competitive depth prediction accuracy. The choice of ResNet-18 or ResNet-50 as the encoder backbone allows flexibility between efficiency and accuracy depending on the available hardware. The multi-scale decoder generates depth predictions at four resolution levels, enabling supervision at multiple scales during training and improved fine-detail recovery during inference.

4.2 Module Descriptions

Module ID	Module Name	Description	Technologies used
M1	Simulator Module	CARLA simulator configuration including server setup, Town04/05 urban map loading, vehicle spawning, and RGB camera sensor attachment at 640x480 resolution with 10 FPS capture rate. Depth ground truth acquired via CARLA LiDAR sensor.	CARLA 0.9.13, Python CARLA API, Pygame for visualization
M2	Data Preprocessing Pipeline	Frame capture from CARLA sensor queue, resizing to 192x640 pixels, normalization using ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225), and temporal triplet grouping for self-supervised pose estimation training.	OpenCV, NumPy, torchvision transforms, PIL

M3	ResNet Encoder	ResNet-18 or ResNet-50 convolutional backbone pretrained on ImageNet, fine-tuned for depth feature extraction. Generates hierarchical feature maps at five scales from input resolution down to 1/32 of input resolution.	PyTorch 2.x, torchvision ResNet, pretrained Monodepth2 weights
M4	Depth Decoder	Multi-scale U-Net style decoder with skip connections from encoder feature maps. Generates depth predictions at four resolution scales (1/4, 1/8, 1/16, 1/32) using sigmoid activation followed by inverse depth conversion.	PyTorch, custom Monodepth2 decoder layers, ELU activation
M5	Pose Estimation Network	Separate pose network implemented as a ResNet-18 encoder that takes concatenated pairs of adjacent frames as input and regresses the 6-DoF relative camera pose between frames. Used during self-supervised training only.	PyTorch, ResNet-18 pose encoder, axis-angle rotation parameterization
M6	Visualization Module	Real-time depth colormap generation using matplotlib plasma or turbo colormaps applied to predicted inverse depth maps. Side-by-side display of RGB frame and depth overlay. Real-time metric display including AbsRel, RMSE, and FPS counter.	Matplotlib, OpenCV, NumPy, Pygame

4.3 Monodepth2 Architecture Detail

The Monodepth2 architecture, proposed by Godard et al. at ICCV 2019, represents the foundational self-supervised monocular depth estimation framework adopted in this project. It consists of three principal components: a depth encoder, a depth decoder, and a pose estimation network. During training, the system learns to predict depth by optimizing photometric consistency between target frames and synthesized views constructed from neighboring frames using the predicted depth and camera motion.

The depth encoder is a standard ResNet architecture, either ResNet-18 (11.7M parameters) or ResNet-50 (23.5M parameters), which processes input RGB images and produces five hierarchical feature representations at decreasing spatial resolutions. These multi-scale features capture both low-level texture and edge information at fine scales and high-level semantic and structural

information at coarse scales. The encoder is initialized with ImageNet pretrained weights to leverage rich visual representations learned from large-scale classification tasks.

The depth decoder takes the five encoder feature maps and progressively upsamples them through a series of convolutional layers and bilinear upsampling operations, with skip connections from the corresponding encoder scale. At each decoder scale, a disparity output is produced using a sigmoid activation function followed by a linear scaling to map the output into the valid disparity range. The final full-resolution depth map is obtained by converting the predicted disparity using the relationship D = 1 / disparity.

4.4 Self-Supervised Loss Function

The self-supervised training of Monodepth2 relies on a photometric reconstruction loss that measures the difference between the original target frame and a synthesized view generated by warping a source frame using the predicted depth and relative camera pose. Given adjacent frames I_{t-1} and I_{t+1} as source frames and I_t as the target frame, the pose network predicts transformations T_{t→t-1} and T_{t→t+1}.

For each source frame I_s, a synthesized target view I_hat_t is generated by projecting pixels from I_s to I_t using the predicted depth D_t and camera intrinsics K. The photometric loss L_ph combines Structural Similarity (SSIM) and L1 error terms, where alpha = 0.85 as in the original Monodepth2 formulation. The minimum reprojection loss mechanism selects the minimum photometric error across all source frames per pixel, effectively masking pixels that are occluded or leave the frame boundary.

An edge-aware smoothness regularization term is added to encourage depth predictions to be smooth in textureless image regions while preserving sharp boundaries at object edges. The total training loss is the sum of the photometric loss and the smoothness term, weighted and summed across all decoder output scales. This multi-scale loss formulation prevents the depth decoder from learning degenerate low-resolution solutions.

4.5 CARLA Integration

The CARLA simulator provides a high-fidelity urban driving simulation environment with realistic vehicle physics, traffic agents, and configurable weather and lighting conditions. The integration with the depth estimation system is implemented through the Python CARLA API, which enables programmatic control of all simulation parameters.

The integration workflow begins with CARLA server initialization and connection via the Python client library. A Town04 or Town05 urban map is loaded to provide a rich set of road geometries, intersections, and urban features. An ego vehicle is spawned at a randomized start location, and an autopilot behaviour planner is enabled to provide realistic driving trajectories without requiring manual control. An RGB camera sensor is attached to the vehicle at a forward-facing position with a 640x480 pixel resolution at ten frames per second, matching the KITTI dataset image dimensions.

During operation, the CARLA sensor callback function places captured frames into a synchronized queue, from which the depth estimation pipeline retrieves and processes frames in real time. A depth camera sensor is simultaneously attached at the same vehicle mounting position to provide ground truth depth measurements for quantitative evaluation.

CHAPTER 5: DATASET / DATA COLLECTION AND FEASIBILITY

5.1 Datasets Used

5.1.1 Primary Dataset: CARLA Synthetic Data

The primary dataset for this project is collected directly from the CARLA autonomous driving simulator with custom map configurations, controlled lighting conditions, and varied traffic density. Data collection involves spawning the ego vehicle in Town04 and Town05 maps and recording synchronized RGB frames, depth maps, and ego-motion pose data as the vehicle navigates autonomously. The CARLA synthetic dataset provides ground-truth pixel-level depth from the simulator's depth camera, synchronized temporal frame sequences required for self-supervised training, and configurable environmental parameters enabling data collection across diverse conditions.

Data is collected at ten frames per second over multiple driving sessions of approximately fifteen minutes each, yielding approximately nine thousand frames per session. Multiple sessions are conducted across different map regions and traffic densities to ensure scene diversity. The total synthetic dataset comprises approximately twenty-seven thousand frames, split into training, validation, and test subsets following a stratified scene-level split of seventy, fifteen, and fifteen percent respectively.

5.1.2 Secondary Dataset: KITTI Benchmark

The KITTI dataset is the primary standard benchmark for autonomous driving perception research and serves as the principal evaluation dataset for this project. Collected by Geiger et al. using a research vehicle equipped with stereo cameras and LiDAR in Karlsruhe, Germany, the KITTI dataset provides 61 driving scenes with 93,000 training images and 697 test frames in the Eigen split formulation. Synchronized LiDAR depth maps are available for evaluation, enabling quantitative comparison of depth estimation accuracy against ground truth.

This project uses the KITTI Eigen split, which is the standard evaluation protocol for self-supervised depth estimation methods and enables direct comparison with published results from Monodepth2 and subsequent works. Pretrained Monodepth2 weights trained on the KITTI Eigen split are used as initialization for the system, providing a well-validated starting point before fine-tuning on CARLA synthetic data.

5.1.3 Reference Dataset: NYU Depth V2

The NYU Depth V2 dataset, containing indoor RGB-D images captured by a Microsoft Kinect sensor, is used exclusively for qualitative comparison of model generalization across significantly different domain environments. The NYU Depth V2 dataset is not used for training or primary evaluation in this project.

5.2 Data Preprocessing Pipeline

1	Frame Acquisition	Capture 640x480 RGB frames from CARLA sensor at 10 FPS using synchronized callback queue	10 FPS, 640x480 resolution
2	Resize	Resize frames from 640x480 to 192x640 using bilinear interpolation to match Monodepth2 training input size	Target: 192x640 pixels
3	Normalize	Convert pixel values from [0, 255] to [0.0, 1.0] and apply ImageNet channel normalization using mean = [0.485, 0.456, 0.406] and std = [0.229, 0.224, 0.225]	ImageNet statistics
4	Augmentation	Apply random horizontal flip, color jitter (brightness ±0.2, contrast ±0.2, saturation ±0.2, hue ±0.1) for training data diversity. Augmentation is disabled during evaluation.	Training only
5	Triplet Grouping	Group consecutive frames into temporal triplets (I_{t-1}, I_t, I_{t+1}) for self-supervised pose estimation training. Static camera periods are filtered out.	Temporal window: 3 frames

5.3 Hardware Requirements

CPU	Intel Core i5 (8th Gen) or AMD Ryzen 5 3600	Intel Core i7 (10th Gen+) or AMD Ryzen 7 5800X
GPU	NVIDIA GTX 1650 (4GB VRAM) — minimum for inference	NVIDIA RTX 2060 or RTX 3060 (8GB VRAM) — for training
RAM	16 GB DDR4 system memory	32 GB DDR4 for CARLA + training workloads
Storage	256 GB SSD (KITTI dataset + CARLA)	512 GB NVMe SSD for fast dataset I/O
OS	Ubuntu 20.04 LTS or Windows 10/11 (64-bit)	Ubuntu 20.04 LTS (preferred for CARLA compatibility)

5.4 Software Requirements

Simulation	CARLA Simulator, Python CARLA API, Pygame	CARLA 0.9.13, Pygame 2.1+
Deep Learning	PyTorch, torchvision, Monodepth2 pretrained weights	PyTorch 2.0+, CUDA 11.8+
Computer Vision	OpenCV, NumPy, Pandas, Matplotlib, PIL	OpenCV 4.8+, NumPy 1.24+
Dev Environment	Python, Jupyter Notebook, VS Code / PyCharm	Python 3.8+
Version Control	Git / GitHub, DVC (Data Version Control), Conda	Git 2.x, Conda 23.x

5.5 Feasibility Analysis

5.5.1 Technical Feasibility

The proposed system is technically feasible based on the availability of mature, well-validated open-source components including the Monodepth2 codebase, CARLA simulator, PyTorch framework, and KITTI dataset. All components have been successfully used in peer-reviewed research and provide documented APIs and pretrained model weights. The target inference performance of ten frames per second is achievable on mid-range GPU hardware using optimized PyTorch inference with mixed-precision computation. ResNet-18 based Monodepth2 achieves approximately twenty-five frames per second on an RTX 3060, well exceeding the ten FPS target.

5.5.2 Economic Feasibility

The project requires no specialized hardware beyond standard computing equipment available in the department laboratory. The CARLA simulator, Monodepth2 codebase, PyTorch framework, and KITTI dataset are all publicly available at no cost. Cloud computing resources such as Google Colab Pro or AWS EC2 GPU instances provide cost-effective alternatives for training workloads at approximately one to three dollars per GPU-hour, making extended training runs economically feasible within the project budget.

5.5.3 Operational Feasibility

The proposed system operates within well-established software environments familiar to the project team, including Python, PyTorch, and Linux. The modular pipeline architecture allows incremental development and testing of individual components before full integration. The team has acquired foundational knowledge of deep learning, computer vision, and autonomous systems through coursework and supplementary study of the Monodepth2 paper and CARLA documentation.

5.6 Risks and Mitigation Strategies

CARLA simulator crashes or version incompatibilities with Python CARLA API	High	Pin CARLA version 0.9.13; maintain a Docker container with fixed dependencies; maintain fallback recorded CARLA data.	Shashank Kumar
Monodepth2 model does not generalize to CARLA synthetic domain after pretrained initialization	High	Fine-tune on CARLA synthetic data for minimum 10 epochs; apply domain randomization during data collection; evaluate cross-domain metrics after each fine-tuning stage.	Sphurti Patil
GPU memory insufficient for training on available hardware	High	Use mixed-precision (FP16) training; reduce batch size to 8; use gradient checkpointing; leverage Google Colab Pro or AWS EC2 GPU instances for large training runs.	Pushkar Raj

Frame rate below ten FPS target during real-time CARLA integration	Medium	Optimize inference pipeline with TorchScript compilation; use ResNet-18 encoder instead of ResNet-50; implement asynchronous sensor data queue to decouple capture from inference.	Shravani V
Ground truth depth from CARLA is sparse or noisy in certain scenarios	Medium	Apply LiDAR point cloud densification using morphological dilation; use multi-frame depth aggregation to fill sparse regions; augment with KITTI validation metrics.	Shashank Kumar
Team member unavailability during critical implementation phases	Low	Shared GitHub repository with documented code; cross-training on each module; weekly team check-ins with documented progress; every module has primary and backup owner.	All Members

CHAPTER 6: EVALUATION PLAN

6.1 Evaluation Metrics

The evaluation of the proposed monocular depth estimation system follows the standard benchmark protocol established by the Eigen split evaluation framework, which is universally adopted in the monocular depth estimation literature. The following quantitative metrics are computed on depth predictions against ground truth depth values from the KITTI Eigen test split and CARLA synthetic depth maps. All metrics are capped at a maximum depth of eighty meters following standard practice.

6.1.1 Absolute Relative Error (AbsRel)

The Absolute Relative Error measures the mean ratio of the absolute depth prediction error to the ground truth depth across all valid pixels. It provides a scale-invariant measure of relative depth accuracy that is independent of the absolute depth range. A lower AbsRel indicates better performance. The target value for this project is AbsRel < 0.115, consistent with the original Monodepth2 performance on the KITTI Eigen benchmark.

6.1.2 Root Mean Square Error (RMSE)

The Root Mean Square Error measures the square root of the average squared difference between predicted and ground truth depth values. RMSE penalizes large errors more heavily than AbsRel, making it sensitive to outlier depth predictions that may represent dangerous estimation failures in autonomous driving contexts. The target RMSE for this project is below 4.8 meters.

6.1.3 Squared Relative Error (SqRel)

The Squared Relative Error is similar to AbsRel but applies a quadratic penalty to relative depth errors, amplifying the impact of large prediction errors in high-depth regions. It is particularly sensitive to errors on distant objects, which are critical for highway driving scenarios. The target

SqRel for this project is below 0.9, consistent with competitive self-supervised depth estimation performance on the KITTI benchmark.

6.1.4 Delta Accuracy Thresholds ($\delta < 1.25$, $\delta < 1.25^2$, $\delta < 1.25^3$)

Delta accuracy metrics measure the percentage of predicted depth pixels for which the ratio between predicted and ground truth depth falls within a threshold factor of 1.25. Three threshold levels are evaluated, corresponding to increasingly strict accuracy requirements. Higher delta accuracy values indicate better performance. The target for $\delta < 1.25$ is above 0.850, meaning at least 85% of predicted depth pixels fall within a 25% error margin of ground truth.

6.1.5 Log RMSE

Log RMSE computes the RMSE in logarithmic depth space, which provides a scale-invariant error measure that treats errors proportionally across different depth ranges. It is particularly useful for evaluating depth estimation performance when both near and far depth ranges are of interest. The target Log RMSE for this project is below 0.190.

6.2 Target Performance Summary

Absolute Relative Error (AbsRel)	Mean relative error across all depth pixels	< 0.115	0.115 (KITTI Eigen)
RMSE	Root mean square depth error in meters	< 4.8 m	4.701 m (KITTI Eigen)
Squared Relative Error (SqRel)	Quadratic relative error metric	< 0.9	0.803 (KITTI Eigen)
delta < 1.25	% pixels within 25% factor of ground truth	> 0.850	0.877 (KITTI Eigen)
Log RMSE	RMSE in logarithmic depth space	< 0.190	0.187 (KITTI Eigen)
Inference Speed	Frames processed per second on evaluation GPU	> 10 FPS	~25 FPS on RTX 3060 (ResNet-18)

6.3 Evaluation Protocol

Quantitative evaluation is conducted on the KITTI Eigen test split comprising 697 frames from 29 scenes held out from the training data. All frames are evaluated at 640x192 resolution with depth predictions scaled using the median scaling procedure standard in self-supervised evaluation, which aligns predicted scale-ambiguous depths with ground truth metric scale using the ratio of ground truth to predicted median depth. Ground truth depth values from the KITTI LiDAR are capped at eighty meters, and pixels with missing ground truth depth are excluded from metric computation.

Additionally, qualitative evaluation is performed on CARLA synthetic frames by visual inspection of depth map overlay quality, boundary sharpness, and temporal consistency across video sequences. Runtime performance evaluation measures frames per second under sustained inference conditions on the target GPU hardware.

CHAPTER 7: WORK PLAN AND PHASE-II TIMELINE

7.1 Phase-I Summary

Phase-I of the project focused on problem definition, literature survey, system architecture design, and preliminary setup. The following activities were completed during Phase-I: comprehensive literature review of 25+ papers in monocular depth estimation, system architecture finalization based on Monodepth2, CARLA simulator installation and configuration, KITTI dataset download and preprocessing scripts development, and preliminary inference testing with pretrained Monodepth2 weights on sample KITTI frames.

7.2 Phase-II Plan

1	CARLA Data Collection	Collect synchronized RGB and depth frames in Town04/05 maps under varied lighting and traffic conditions; implement data logging pipeline.	Weeks 1-2	Shashank, Pushkar
2	Preprocessing Pipeline Implementation	Implement and test frame resizing, normalization, augmentation, and temporal triplet grouping modules; validate on sample CARLA data.	Weeks 2-3	Shravani V
3	Fine-Tuning on CARLA Data	Fine-tune pretrained Monodepth2 ResNet-18 on CARLA synthetic dataset; monitor training convergence; evaluate on CARLA validation split.	Weeks 3-5	Sphurti, Pushkar
4	KITTI Benchmark Evaluation	Evaluate fine-tuned and pretrained models on KITTI Eigen test split; compute AbsRel, RMSE, delta accuracy, Log RMSE; compare with published baselines.	Weeks 5-6	Shashank Kumar
5	CARLA Real-Time Integration	Implement complete CARLA integration pipeline; achieve real-time depth inference at 10+ FPS; test visualization module with colorized depth overlays.	Weeks 6-8	Shravani, Sphurti

6	Performance Optimization	Profile inference pipeline; apply TorchScript optimization; reduce latency below 100 ms; implement async sensor queue for stable frame rate.	Weeks 8-9	Pushkar Raj
7	Ablation Studies	Compare ResNet-18 vs ResNet-50 encoders; evaluate impact of fine-tuning; analyze loss function components; document findings.	Weeks 9-10	All Members
8	Conference Paper Writing	Draft and refine IEEE-format conference paper documenting system design, methodology, experimental results, and conclusions.	Weeks 10-12	All Members
9	Phase-II Report Preparation	Compile complete Phase-II project report with experimental results, implementation details, evaluation analysis, and future work.	Weeks 12-13	All Members
10	Project Demonstration	Prepare and deliver live demonstration of the CARLA-integrated real-time depth estimation system for project evaluation panel.	Week 14	All Members

CHAPTER 8: CONFERENCE PAPER STATUS

8.1 Research Paper Details

Paper Title	Real-Time Monocular Depth Estimation for Autonomous Driving Using Self-Supervised Deep Learning and CARLA Simulator Integration
Authors	Sphurti Patil, Shravani V, Shashank Kumar, Pushkar Raj, Mr. Manjunath S.R (Guide)
Target Venue	IEEE International Conference on Intelligent Transportation Systems (ITSC 2025) / International Conference on Computer Vision and Image Processing (CVIP 2025)
Paper Type	Conference Paper (IEEE Format, 6 pages)
Current Status	In Progress — Literature Survey and Methodology sections drafted; experimental results pending Phase-II implementation
Target Submission Date	To be finalized during Phase-II (Weeks 10-12 of Phase-II plan)

8.2 Paper Overview

The research paper associated with this project documents the design, implementation, and evaluation of a real-time monocular depth estimation system for autonomous driving based on self-supervised deep learning. The paper presents the complete system architecture, CARLA integration methodology, experimental evaluation results on the KITTI Eigen benchmark, and a comparative analysis against published Monodepth2 performance baselines.

Key contributions described in the paper include the design and validation of an end-to-end CARLA-integrated monocular depth estimation pipeline, an empirical analysis of domain adaptation from CARLA synthetic training data to KITTI real-world evaluation, and a systematic performance analysis of ResNet-18 and ResNet-50 encoder variants for real-time autonomous driving applications on commodity hardware.

The paper will be prepared in IEEE two-column conference format with a minimum of six pages, including an abstract, introduction, related work, methodology, experimental results, and

conclusion sections. Experimental results and quantitative evaluation tables will be included once Phase-II implementation is complete.

CHAPTER 9: CONCLUSION AND FUTURE WORK

9.1 Conclusion

This report presents the Phase-I work for the project titled "Real-Time Monocular Depth Estimation for Autonomous Driving Using Deep Learning," which aims to develop a practical and validated monocular depth estimation pipeline for autonomous driving applications using the Monodepth2 self-supervised deep learning architecture integrated with the CARLA autonomous driving simulator.

Phase-I activities have established a solid foundation for the project through a comprehensive literature survey of twenty-five papers spanning the development of monocular depth estimation from early CNN-based supervised approaches to recent transformer-based foundation models. The literature survey has identified key research gaps including the need for real-time inference capability, robust self-supervised training without LiDAR annotations, and practical simulation-to-real adaptation strategies.

The proposed system architecture has been designed around the well-validated Monodepth2 framework, with a modular pipeline encompassing CARLA simulation data collection, preprocessing, ResNet encoder feature extraction, multi-scale depth decoding, and real-time visualization. The self-supervised photometric consistency loss formulation eliminates the need for expensive ground-truth depth annotations while enabling competitive depth estimation accuracy on standard benchmarks.

The feasibility analysis confirms that the proposed system is technically, economically, and operationally feasible within the project constraints. Hardware and software requirements have been clearly defined, and risk mitigation strategies have been established for the most critical implementation challenges including CARLA version compatibility, model domain adaptation, GPU memory constraints, and real-time inference performance.

9.2 Expected Outcomes

- A fully functional real-time monocular depth estimation pipeline achieving depth inference at ten or more frames per second on commodity GPU hardware.
- Quantitative evaluation results on the KITTI Eigen benchmark demonstrating Absolute Relative Error below 0.115 and RMSE below 4.8 meters.
- A validated CARLA integration demonstrating real-time depth map overlay on simulated autonomous driving video streams.
- A research paper documenting the system design, experimental methodology, and evaluation results for submission to an IEEE conference.
- A comprehensive Phase-II report with full implementation details, experimental analysis, and reproducible code repository.

9.3 Future Work

Several research directions are identified for extending the current work beyond the scope of this project:

Domain Adaptation: The primary future direction is improving generalization of the CARLA-trained model to real-world KITTI and nuScenes driving sequences. Domain adaptation techniques including adversarial training, style transfer, and domain randomization are promising approaches to bridge the sim-to-real gap.

Dynamic Object Handling: Current self-supervised methods including Monodepth2 struggle with depth estimation accuracy on dynamic objects such as moving vehicles and pedestrians. Future work will investigate semantic masking and object-aware training strategies.

Metric Scale Recovery: Future research will explore integration of velocity supervision, accelerometer data, or sparse LiDAR points as auxiliary scale cues to enable metric-scale depth prediction without full LiDAR supervision.

Adverse Condition Robustness: Future work will investigate the performance degradation of monocular depth estimation under challenging environmental conditions including nighttime driving, rain, fog, and direct sunlight.

Multi-Task Learning: Integration of monocular depth estimation with related perception tasks including semantic segmentation, object detection, and ego-motion estimation in a unified multi-task learning framework is a natural extension.

Hardware-Aware Optimization: Future work will investigate knowledge distillation, neural architecture search, and hardware-aware optimization techniques to reduce model size and inference latency for deployment on embedded automotive hardware including Jetson Nano platforms.

REFERENCES

[1] D. Eigen, C. Puhrsch, and R. Fergus, "Depth map prediction from a single image using a multi-scale deep network," in Advances in Neural Information Processing Systems (NeurIPS), vol. 27, pp. 2366–2374, 2014.

[2] R. Garg, V. K. BG, G. Carneiro, and I. Reid, "Unsupervised CNN for single view depth estimation: Geometry to the rescue," in Proc. European Conference on Computer Vision (ECCV), Springer, pp. 740–756, 2016.

[3] I. Laina, C. Rupprecht, V. Belagiannis, F. Tombari, and N. Navab, "Deeper depth prediction with fully convolutional residual networks," in Proc. IEEE International Conference on 3D Vision (3DV), pp. 239–248, 2016.

[4] C. Godard, O. Mac Aodha, M. Firman, and G. J. Brostow, "Digging into self-supervised monocular depth estimation," in Proc. IEEE/CVF International Conference on Computer Vision (ICCV), pp. 3828–3838, 2019.

[5] J. H. Lee, M. Han, D. W. Ko, and I. H. Suh, "From big to small: Multi-scale local planar guidance for monocular depth estimation," arXiv preprint arXiv:1907.10326, 2019.

[6] V. Guizilini, R. Ambrus, S. Pillai, A. Raventos, and A. Gaidon, "3D packing for self-supervised monocular depth estimation," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2485–2494, 2020.

[7] S. F. Bhat, I. Alhashim, and P. Wonka, "AdaBins: Depth estimation using adaptive bins," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4009–4018, 2021.

[8] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision transformers for dense prediction," in Proc. IEEE/CVF International Conference on Computer Vision (ICCV), pp. 12179–12188, 2021.

[9] M. Klingner, J. A. Termohlen, J. Mikolajczyk, and T. Fingscheidt, "Self-supervised monocular depth estimation: Solving the dynamic object problem by semantic guidance," in Proc. European Conference on Computer Vision (ECCV), pp. 582–600, 2020.

[10] J. Watson, M. Firman, G. Brostow, and D. Turmukhambetov, "Self-supervised monocular depth hints," in Proc. IEEE/CVF International Conference on Computer Vision (ICCV), pp. 2162–2171, 2019.

[11] R. Zhang, P.-S. Wang, and Z. Li, "Lite-mono: A lightweight CNN and transformer architecture for self-supervised monocular depth estimation," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 18537–18546, 2023.

[12] W. Yin et al., "Metric3D: Towards zero-shot metric 3D prediction from a single image," in Proc. IEEE/CVF International Conference on Computer Vision (ICCV), pp. 9043–9053, 2023.

[13] L. Yang et al., "Depth anything: Unleashing the power of large-scale unlabeled data," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024.

[14] L. Yang et al., "Depth Anything V2," arXiv preprint arXiv:2406.09414, 2024.

[15] X. Hu et al., "Metric3D v2: A versatile monocular geometric foundation model for zero-shot metric depth and surface normal estimation," IEEE Transactions on Pattern Analysis and Machine Intelligence, 2024.

[16] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 3354–3361, 2012.

[17] A. Dosovitskiy et al., "CARLA: An open urban driving simulator," in Proc. Conference on Robot Learning (CoRL), pp. 1–16, 2017.

[18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778, 2016.

[19] N. Silberman, D. Hoiem, P. Kohli, and R. Fergus, "Indoor segmentation and support inference from RGBD images," in Proc. European Conference on Computer Vision (ECCV), pp. 746–760, 2012.

[20] Z. Li et al., "BinsFormer: Revisiting adaptive bins for monocular depth estimation," IEEE Transactions on Image Processing, vol. 33, pp. 1924–1937, 2024.

[21] D. Dong et al., "Towards real-time monocular depth estimation for robotics: A survey," IEEE Transactions on Intelligent Transportation Systems, vol. 23, no. 10, pp. 16940–16961, 2022.

[22] S. F. Bhat et al., "ZoeDepth: Zero-shot transfer by combining relative and metric depth," arXiv preprint arXiv:2302.12288, 2023.

[23] A. Mertan, D. J. Duff, and G. Unal, "Single image depth estimation: An overview," Digital Signal Processing, vol. 123, p. 103441, 2022.

[24] J. Bian et al., "Unsupervised scale-consistent depth and ego-motion learning from monocular video," in Advances in Neural Information Processing Systems (NeurIPS), 2019.

[25] Z. Liu et al., "Swin Transformer: Hierarchical vision transformer using shifted windows," in Proc. IEEE/CVF International Conference on Computer Vision (ICCV), pp. 10012–10022, 2021.

APPENDIX

Appendix A: Project Presentation Slides

The project presentation slides prepared for Phase-I review are attached as a supplementary document to this report. The presentation covers the problem statement, literature survey highlights, proposed methodology, system architecture, dataset plan, evaluation metrics, and Phase-II timeline in a condensed visual format suitable for panel presentation.

Appendix B: Abbreviations and Glossary

AbsRel	Absolute Relative Error — mean ratio of absolute depth prediction error to ground truth depth
CARLA	Car Learning to Act — open-source autonomous driving simulator developed by Intel Labs and CVC
CNN	Convolutional Neural Network — a class of deep neural networks widely used for image processing and computer vision
FPS	Frames Per Second — measure of real-time inference throughput
KITTI	Karlsruhe Institute of Technology and Toyota Technological Institute — standard autonomous driving benchmark dataset
LiDAR	Light Detection and Ranging — sensor technology using laser pulses to measure distances and generate 3D point clouds
NYU Depth V2	New York University Depth Version 2 — indoor RGB-D dataset collected using Microsoft Kinect sensor
RMSE	Root Mean Square Error — square root of mean squared depth prediction error
RGB	Red Green Blue — standard three-channel color image format used as input to the depth estimation network
ResNet	Residual Network — deep convolutional neural network architecture with skip connections, used as encoder backbone
SSIM	Structural Similarity Index Measure — image quality metric used as component of photometric reconstruction loss
ViT	Vision Transformer — transformer-based architecture applied to image patches for visual representation learning

VTU	Visvesvaraya Technological University — affiliating university for JSS Academy of Technical Education, Bengaluru

Appendix C: Code Repository

The project code is maintained in a private GitHub repository accessible to the project guide and evaluation panel upon request. The repository contains the following components:

- CARLA sensor integration scripts for vehicle spawning, camera configuration, and synchronized data capture.
- Data preprocessing pipeline scripts for frame resizing, normalization, augmentation, and dataset split generation.
- Monodepth2 model loading, inference, and depth visualization scripts.
- KITTI evaluation scripts for computing AbsRel, RMSE, SqRel, delta accuracy, and Log RMSE metrics.
- Requirements file (requirements.txt) and Conda environment configuration for reproducible environment setup.

Repository: [To be made available upon Phase-II completion]

Environment: Python 3.8+, PyTorch 2.0+, CUDA 11.8, CARLA 0.9.13, Ubuntu 20.04 LTS